MEMORIA DEL PRODUCTO MÍNIMO VIABLE

Vibra Music

Web de la academia musical — Carmona (Sevilla)

Producción: **https://vibramusicstudio.es**

Cliente: Vibra Music Studio
Entrega: Vibra Music Studio
Versión: 1.0.0 · **Fecha:** 2026

Índice

1. Resumen ejecutivo
2. Objetivo y alcance
3. Funcionalidades
4. Stack tecnológico
5. Arquitectura
6. Modelo de datos
7. Autenticación y correo
8. Sistema de diseño
9. Despliegue e infraestructura
10. Manual de uso
11. Estado y limitaciones
12. Roadmap
13. Anexos

1. Resumen ejecutivo

Vibra Music es una academia de música en Carmona (Sevilla) que ofrece clases particulares y grupales, masterclasses y reserva de estudio de grabación. Este proyecto entrega la **primera versión funcional (PMV)** de su web: una landing pública con toda la información y canales de contacto, más un **backend real** (registro, login, recuperación de contraseña, reservas de estudio y mensajería) y un **panel de administración**.

El objetivo del PMV es validar el flujo completo de negocio —visitante → reserva o mensaje → gestión por el centro— sobre una base técnica sólida, desplegada en producción y lista para recibir contenido definitivo.

2. Objetivo y alcance

Objetivo

Disponer de una web propia, profesional y funcional que centralice la información de la academia y permita gestionar reservas y consultas sin depender de herramientas de terceros.

Incluido en el PMV

- Landing pública con secciones: portada (hero), accesos rápidos, sobre nosotros, cursos, estudio de grabación, masterclasses y contacto.
- Registro e inicio de sesión reales con confirmación por email.
- Recuperación de contraseña por email.
- Perfil de usuario con sus reservas y mensajes.
- Reserva de franjas del estudio (compartidas entre usuarios).
- Formulario de contacto (accesible también sin cuenta).
- Panel de administración para gestionar reservas y mensajes.
- Dominio propio, HTTPS y correo de contacto.

Fuera de alcance (fases posteriores)

- Pasarela de pago / cobro online.
- Contenido real definitivo (catálogo, precios e imágenes finales).
- Panel de administración ampliado (edición de cursos, usuarios, informes).
- Emails transaccionales más allá de la autenticación.

3. Funcionalidades

3.1 Zona pública (visitante)

Función	Descripción
Landing	Secciones informativas con navegación por anclas y scroll suave
Cursos	Catálogo filtrable por categoría, con modalidad y precio
Estudio	Equipamiento, demos de audio y **reserva de franjas**
Masterclasses	Oferta de formaciones intensivas
Contacto	Formulario (nombre, email, mensaje) + datos y redes sociales
Tema	Selector claro/oscuro

3.2 Usuario registrado

Función	Descripción
Registro	Alta con confirmación por email
Login / logout	Sesión persistente con refresco automático
Recuperar contraseña	Envío de email y establecimiento de nueva contraseña
Perfil	Datos de la cuenta, historial de reservas y mensajes
Reservar estudio	Elección de fecha y franja; se impide doble reserva

3.3 Administrador

Función	Descripción
Acceso	Ruta `/admin`, protegida por `authGuard` + `adminGuard`
Reservas	Listado de todas las reservas; cancelación
Mensajes	Listado de todos los mensajes de contacto

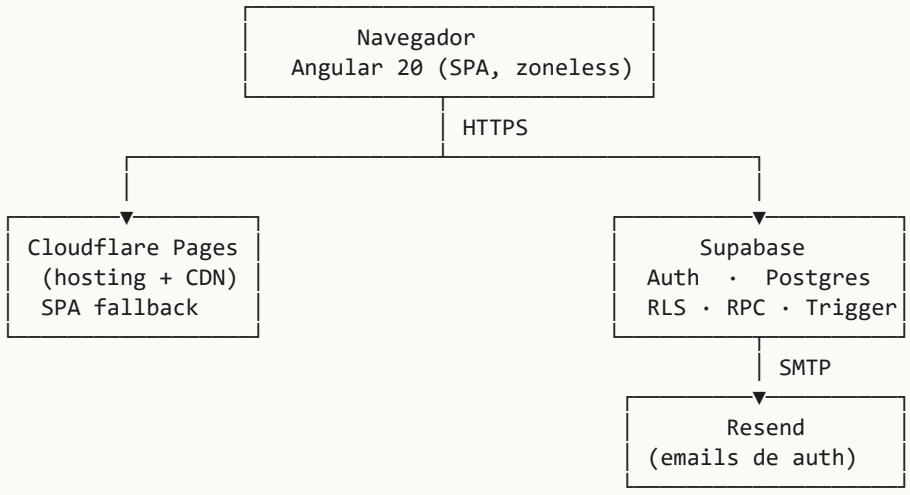
4. Stack tecnológico

Tecnología	Versión	Uso
Angular	^20.3.30	Framework principal (standalone + lazy loading)
Angular CLI / `@angular/build`	^20.3.35	Build y tooling
TypeScript	~5.8.0	Lenguaje
SCSS	—	Estilos (variables, mixins, globales)
RxJS	~7.8.0	Reactividad
Supabase (`@supabase/supabase-js`)	^2.116.0	Auth + PostgreSQL (RLS)
Vitest	^3.2.7	Tests unitarios (77 tests)
Prettier	^3.8.1	Formateo
Cloudflare Pages	—	Hosting y despliegue continuo
Resend (SMTP)	—	Envío de emails de autenticación

Notas de arquitectura

- **Standalone components** sin NgModules; **carga perezosa** en todas las rutas.
- **Change detection zoneless** (`provideZonelessChangeDetection()`) con **signals**.
- **Backend real** en Supabase mediante un cliente compartido y servicios especializados (`auth` , `booking` , `messages` , `admin`).

5. Arquitectura



Estructura del proyecto

```
src/
├── app/
│   ├── app.ts / app.html / app.scss       # Componente raíz
│   ├── app.config.ts                      # Providers (zoneless, router, supabase)
│   ├── app.routes.ts                      # Rutas (lazy)
│   ├── core/
│   │   ├── data/                          # Datos de cursos, masterclasses, estudio
│   │   ├── guards/                        # auth.guard.ts, admin.guard.ts
│   │   ├── models/                        # Interfaces (Course, Booking, Message...)
│   │   └── services/                      # supabase, auth, booking, messages, admin, theme, toast
│   ├── features/
│   │   ├── authentication/                # login, registro, recuperar, reset-password
│   │   ├── landing/                       # home + componentes de sección
│   │   ├── admin/                         # panel de administración
│   │   ├── profile/                       # perfil (protegido)
│   │   └── not-found/                     # 404
│   └── shared/components/                 # button, card, header, footer, toast, avatar, spinner
├── assets/styles/                         # _variables.scss, _mixins.scss, _global.scss
└── environments/                          # Configuración Supabase (prod / dev)
supabase/
├── schema.sql                             # Esquema completo (idempotente)
└── emails/                                # Plantillas HTML de los emails de auth
scripts/
└── postbuild.mjs                          # Genera 404.html (fallback SPA)
```

6. Modelo de datos

Base de datos **PostgreSQL** en Supabase con **Row Level Security (RLS)** activado en todas las tablas. El esquema completo e idempotente está en `supabase/schema.sql`.

6.1 Tablas

Tabla	Campos principales	Descripción
`profiles`	`id`, `name`, `email`, `role`, `created_at`	Una fila por usuario; el rol decide quién es admin
`bookings`	`id`, `user_id`, `date`, `slot`, `service`, `notes`, `created_at`	Reservas del estudio; restricción única `(date, slot)`
`messages`	`id`, `user_id`, `name`, `email`, `message`, `read`, `created_at`	Mensajes del formulario de contacto

6.2 Seguridad (RLS)

Tabla	Operación	Regla
`profiles`	select	El propio usuario o un admin
`profiles`	update	Solo el propio usuario
`bookings`	select	El propio usuario o un admin
`bookings`	insert	Solo en nombre propio
`bookings`	delete	El propio usuario o un admin
`messages`	insert	Cualquiera (incluso sin cuenta)
`messages`	select	El propio usuario (por `user_id` o email) o un admin
`messages`	update	Solo admin

6.3 Funciones y disparadores

Objeto	Tipo	Función
`is_admin()`	Función `security definer`	Comprueba si el usuario actual es admin (evita recursión en RLS)
`handle_new_user()`	Trigger `after insert on auth.users`	Crea el `profile` automáticamente al registrarse
`get_booked_slots(date)`	Función `security definer`	Devuelve **solo** las franjas ocupadas de una fecha, sin datos personales

> Las funciones `security definer` permiten operaciones controladas que el RLS impediría (por ejemplo, conocer la disponibilidad sin exponer reservas ajenas).

7. Autenticación y correo

- **Auth real** con Supabase: registro con **confirmación por email**, login, cierre de sesión y **recuperación de contraseña**.
- Sesión **persistente** (`persistSession` , `autoRefreshToken`); los enlaces de confirmación y de reset se resuelven en la app (`detectSessionInUrl`).
- **Redirecciones configuradas:** confirmación → `/login` ; reset → `/reset-password` .
- **Emails con identidad de marca** (plantillas HTML versionadas en `supabase/emails/`), enviados mediante **SMTP propio con Resend** desde `Vibra Music <support@vibramusicstudio.es>` .
- **Correo de contacto** (`support@vibramusicstudio.es`) recibido por **Cloudflare Email Routing** (gratuito).

Configuración en Supabase (Auth → URL Configuration)

Parámetro	Valor
Site URL	`https://vibramusicstudio.es`
Redirect URLs	`https://vibramusicstudio.es/**` (cubre `/login` y `/reset-password`)

Configuración SMTP (Auth → SMTP Settings)

Campo	Valor
Host	`smtp.resend.com`
Puerto	`465` (SSL)
Usuario	`resend`
Contraseña	API Key de Resend
Remitente	`support@vibramusicstudio.es`
Nombre remitente	`Vibra Music`

8. Sistema de diseño

- **Fuente:** Afacad Flux (con fallback Arial/Helvetica).
- **Color primario:** rojo `#e73938`. **Fondo:** blanco cálido `#f6f5f2`.
- Tema **claro/oscuro** seleccionable, con tokens centralizados (`_variables.scss`, `_mixins.scss`, `_global.scss`).
- **Responsive** con breakpoints 576 / 768 / 992 / 1200 px; contenedor máx. 1024 px.
- Estilos **100% SCSS propios**, sin frameworks CSS externos.

9. Despliegue e infraestructura

Elemento	Detalle
Hosting	Cloudflare Pages (Git integrado, SSL automático)
Rama de producción	`dev` (cada push lanza build y deploy)
Build	`npm run build` → `dist/vibramusic/browser` (`NODE_VERSION=22`)
Fallback SPA	`postbuild` copia `index.html` como `404.html`
Dominio	`vibramusicstudio.es` (apex); `www` → **301** a la raíz
DNS	Zona gestionada por Cloudflare
Correo entrante	Email Routing para `support@vibramusicstudio.es`
Límite	Cloudflare Pages admite **máx. 25 MiB por fichero** (imágenes optimizadas)

Rutas de la aplicación

Ruta	Página	Guard
`/`	Landing (home)	—
`/login`	Login	—
`/registro`	Registro	—
`/recuperar`	Solicitar recuperación de contraseña	—
`/reset-password`	Establecer nueva contraseña	—
`/perfil`	Perfil del usuario	`authGuard`
`/admin`	Panel de administración	`authGuard` + `adminGuard`
`**`	Página 404	—

10. Manual de uso

10.1 Usuario

1. **Crear cuenta:** `/registro` → rellenar nombre, email y contraseña → recibir email de confirmación → confirmar → iniciar sesión en `/login`.
2. **Iniciar sesión:** `/login`. La sesión se mantiene al volver a la web.
3. **Recuperar contraseña:** `/recuperar` → introducir email → abrir el enlace del correo → definir la nueva contraseña en `/reset-password`.

4. **Reservar el estudio:** en la sección *Estudio*, elegir fecha y franja libre → confirmar. Las franjas ocupadas aparecen bloqueadas.
5. **Enviar consulta:** sección *Contacto* → rellenar y enviar (no requiere cuenta).
6. **Perfil:** `/perfil` muestra los datos de la cuenta y el historial de reservas y mensajes.

10.2 Administrador

1. El rol admin se asigna en la base de datos: `update public.profiles set role = 'admin' where email = '...';`
2. **Panel:** `/admin` (solo usable con rol admin).
3. **Reservas:** listado completo; permite cancelar reservas.
4. **Mensajes:** listado de todas las consultas de contacto.

11. Estado del PMV, limitaciones y contenido pendiente

- **Backend y autenticación:** reales y funcionales (Supabase).
- **Tests:** 77 tests unitarios en verde (Vitest).
- **Contenido pendiente de datos definitivos:** catálogo de cursos y precios, imágenes y datos de contacto deben validarse o sustituirse por la información real del cliente antes del lanzamiento.
- **Sin pasarela de pago**: las reservas/inscripciones no generan cobro.
- **Imágenes**: las definitivas deben respetar el límite de 25 MiB/fichero de Cloudflare Pages (usar JPEG/WebP optimizados).

12. Roadmap / siguientes pasos

1. Reemplazar contenido placeholder por el contenido final del cliente.
2. Optimizar y subir las imágenes definitivas.
3. Ampliar el panel de administración (gestión de cursos, usuarios, métricas).
4. Valorar pasarela de pago para inscripciones.
5. Analítica de uso y mejora continua de SEO/accesibilidad.

13. Anexos

- **Repositorio:** https://github.com/CharliePl8/vibramusic
- **Producción:** https://vibramusicstudio.es
- **Esquema de datos:** `supabase/schema.sql`
- **Plantillas de email:** `supabase/emails/`
- **Licencia:** propietaria — © Vibra Music Studio (ver `LICENSE`).